Military Institute of Science and Technology
Department of Computer Science and Engineering

Peripheral/Computer Connections & Interrupt

Course Name: Computer Interfacing
Course Code: CSE-405

Book Reference

Computer Peripherals (3rd Edition) - Cook and White; Butterworth-Heinemann (1995)

Introduction

- Computer instructions are taken from memory and executed in the CPU
- It involves requests for data from memory and sending results back to memory.
- Part of the computer's memory consist of Read Only Memory (ROM) where a program is available when the machine is first switched ON.
- This program reads other programs into RAM, known as bootstrapping.
- Special instructions are available for accessing the peripheral interfaces.

Component of a Digital Computer

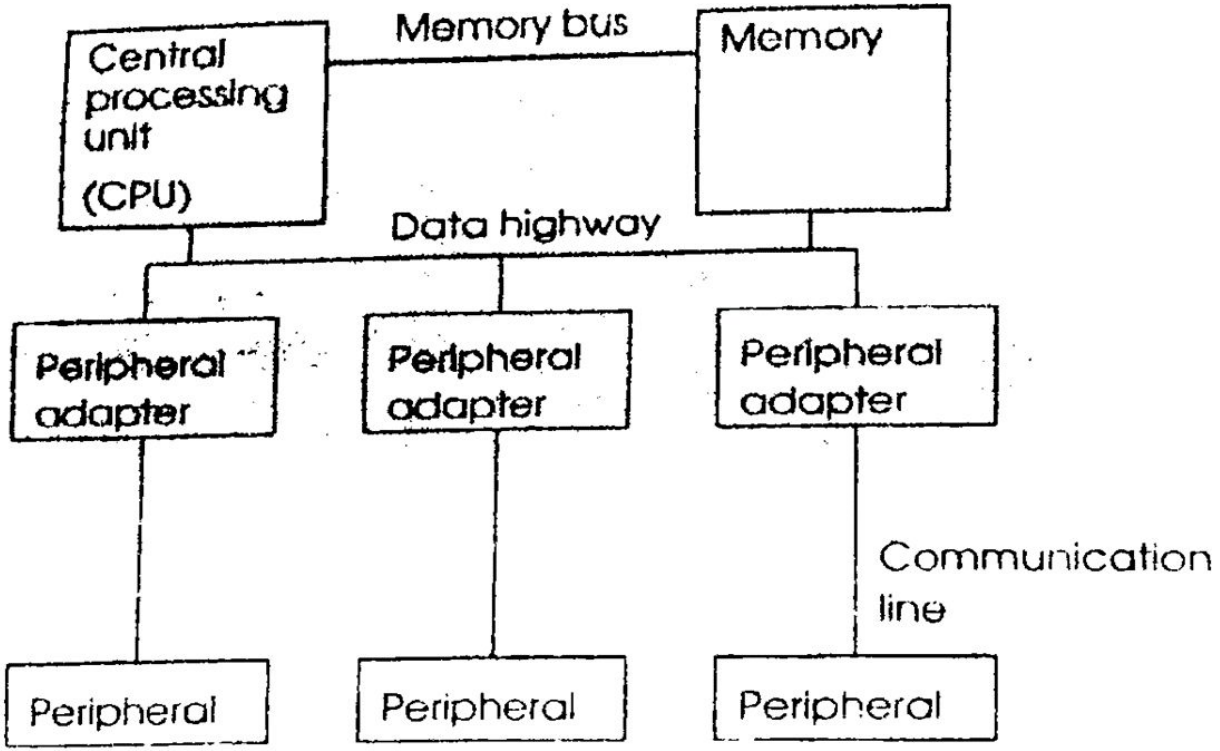


Figure 2.1 *Components of a digital computer*

Data Highway

Data Highway

- Data (including instructions) are moved around the computer on a set of wires forming a data highway (Bus).
  - Data Bus
  - Address Bus
  - Control Bus

Data Highway: Data Bus

- It contains the information or program instructions required by the CPU.
- The speed of operation of a computer depend on the rate at which data can be made available
- A good way to improve the speed is to add more data lines to transfer more data at a time.
- Small systems use 8-bit data bus. They are relatively slow but of low cost.
- 8086 uses 16-bit and 80386 uses 32-bit bus.

Data Highway: Address Bus

- It contains address information.
- The number of lines available for this determines the maximum amount of physical memory that can be addressed.
- If there are n lines , then 2^n locations can be addressed.
- Small system have 16 address line - 2^{16}= 65536 locations
- 8086 has 20 address lines and thus it can address 2^{20}=1048576 (1 Mega bytes) memory locations

Data Highway: Control Bus

- It contains control signals.
- Control signals are required to maintain orderly flow of data along the bus.
- It is necessary to indicate whether to **read** or **write** data during a transfer.
- It also indicates how much data to transfer, timing signals, status lines etc.
- It also indicates whether a transfer is to **memory** or a **peripheral adaptor**

Peripheral Adapter Register

Peripheral Adapter Register

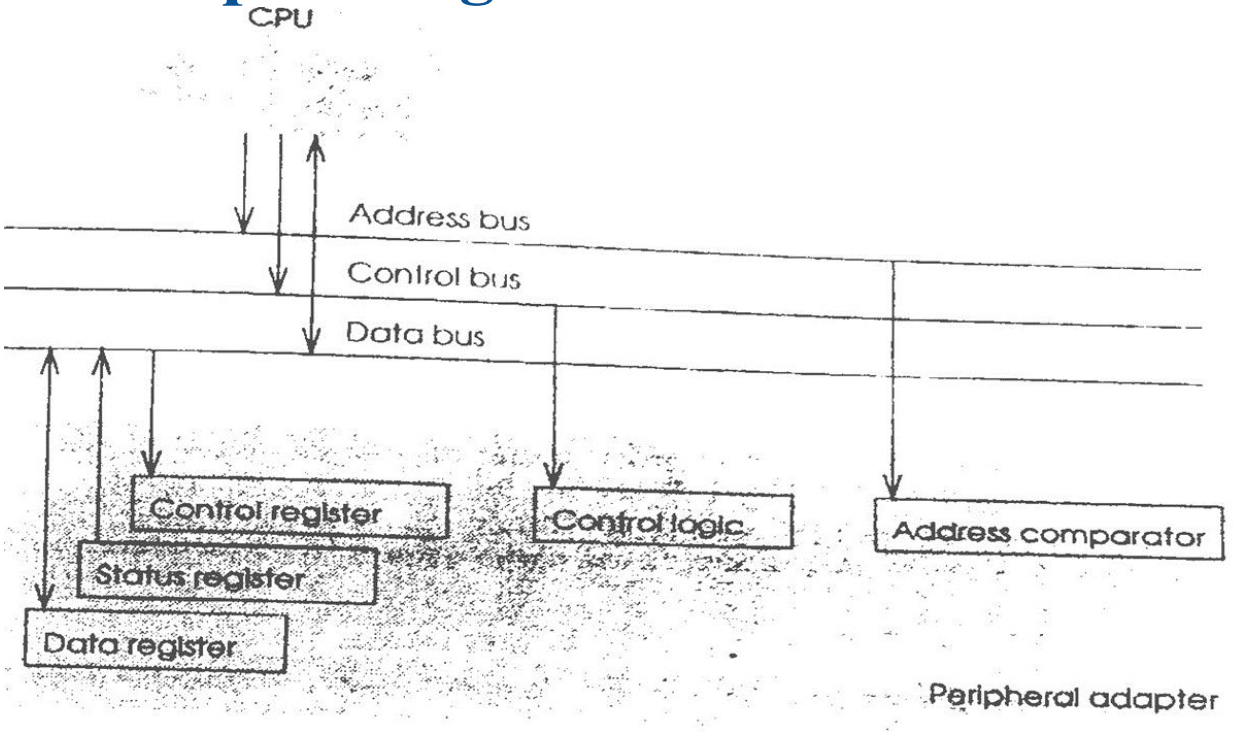


Figure 2.2 *Connection of peripheral adapter to highway*

Peripheral Adapter Register

Peripheral adaptors are directly connected to the buses.

PA contains:

- Address Comparator.
- Control register
- Status register
- Data register
- Control Logic

Peripheral Adapter Register

❖ **Address Comparator**

- It is necessary to distinguish between adapters that are used for different peripherals.
- Address lines are compared with a values allocated to the adaptor
- An adaptor may recognize a single address or a small group of address.
- It compares the address on the address bus with the peripheral adapter's address.

❖ **Control logic**

- If the address on the bus matches the adapter address then the control lines are interpreted by the ***control logic*** to perform the required function.
- It is connected to a data bus typically to read from or write to a ***register*** (may be data register)

Peripheral Adapter Register

- **Control Register**
  - Stores values written to it to control the operation of the adaptor (Receive, transmit, interrupt, mode).
- **Status Register**
  - Can be read by the CPU to determine the status of the device (e.g. w**hether it is ready for use, busy, error, buffer overflow, transmission complete, receive complete etc.**)
  - Each piece of information stored in the registers usually needs only a single bit
  - Several such bits are stored in each register, each bit is often known as a *flag*.
- **Data Register**
  - Used to **hold temporarily a value** to be transferred to or from the peripheral
  - So that it is not necessary to synchronize the computer with the peripheral.

Computer Input/Output Operation: Programmed I/O

Programmed I/0

- The programmed I/O was the simplest type of I/O technique for the exchanges of data or any types of communication **between the processor and the external devices.**
- With programmed I/O, data are exchanged between the processor and the I/O module.
- The instruction set for the CPU typically contains instructions such as
  - **Input data to the CPU from the peripheral**
  - **Output data from the CPU to the peripheral**
  - **Set individual bits in the peripheral adapter's control register**
  - **Test individual bits in the peripheral adapter's status register**

Programmed I/0 : Output

- Output of the data item is straightforward
- The CPU addresses the adapter and tests status register.
- If the relevant bit in the register shows that the device is ready then CPU puts data into the peripherals.
- If the peripheral is not ready, then the program must wait and try again.

Programmed I/0 : Input

- For data input, the status of the input device may be tested and data taken from the input data register if it is available.
- If a number of peripherals are active at the same time, it is necessary to test each one for readiness.
- The technique of testing a number of peripherals in turn is known as ***software polling.***

Programmed I/0 : Example

- Processor instructions for communicating with peripherals are often given obvious mnemonics such as "IN" and "OUT"

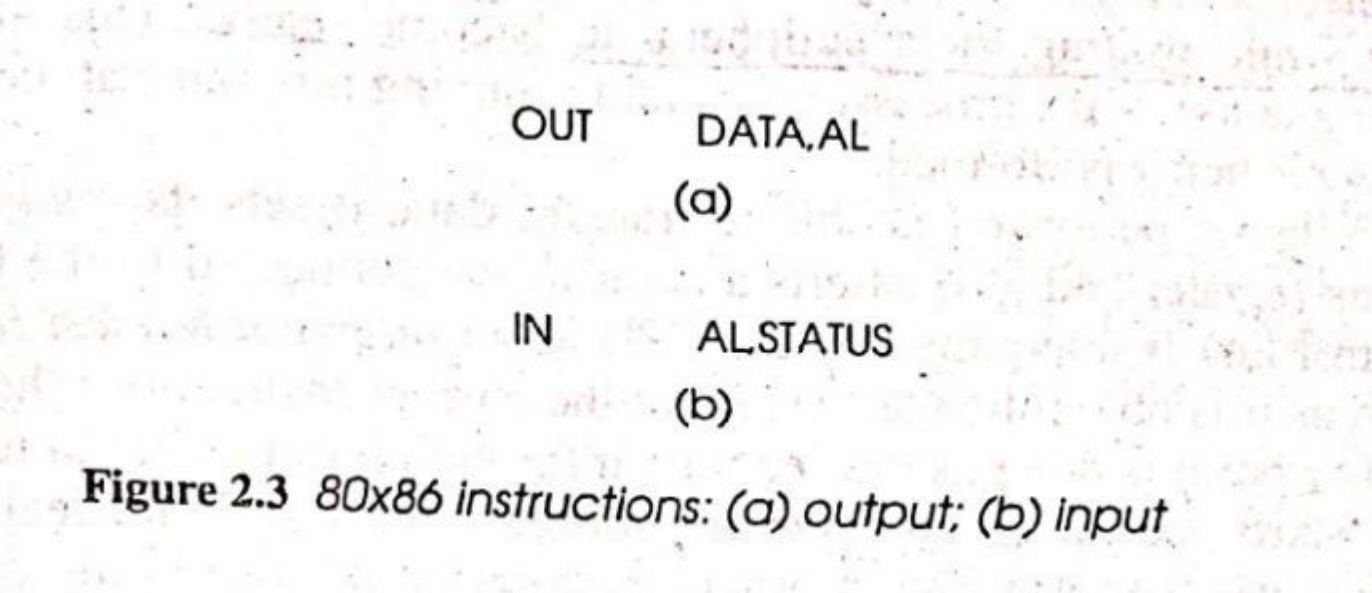
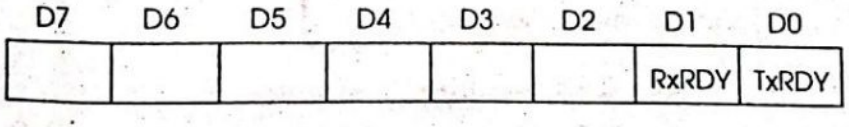


Figure 2.3 *80x86 instructions: (a) output; (b) input*

Programmed I/0 : Example

- Processor instructions for communicating with peripherals are often given obvious mnemonics such as "IN" and "OUT"



(a)

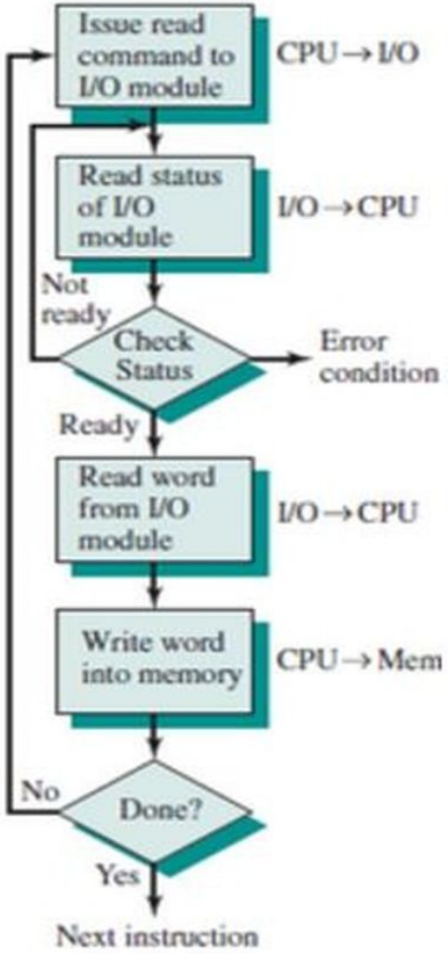
```
STATUS   EQU   0FFH          ; Address of status register
DATA     EQU   0FEH          ; Address of data register

CHECK:   IN    AL,STATUS     ; Get status register
         TEST  AL,1B         ; Test TxRDY bit
         JZ    CHECK         ; Repeat until ready
         MOV   AL,CHARACTER  ; Get the character to send
         OUT   DATA,AL       ; Output to the data register
                             ;  for transmission
```

(b)

Figure 2.4 *Transmitting a character via an 8251; (a) schematic view of the status register; (b) program used*

Programmed I/0 : Data Transfer

- In this case, the I/O device does not have direct access to the memory unit.
- A transfer from I/O device to memory requires the execution of several instructions by the CPU, including an input instruction to transfer the data from device to the CPU and store instruction to transfer the data from CPU to memory.
- In programmed I/O, the CPU stays in the program loop until the I/O unit indicates that it is ready for data transfer.
- This is a time consuming process since it needlessly keeps the CPU busy.



Computer Input/Output Operation: Interrupt

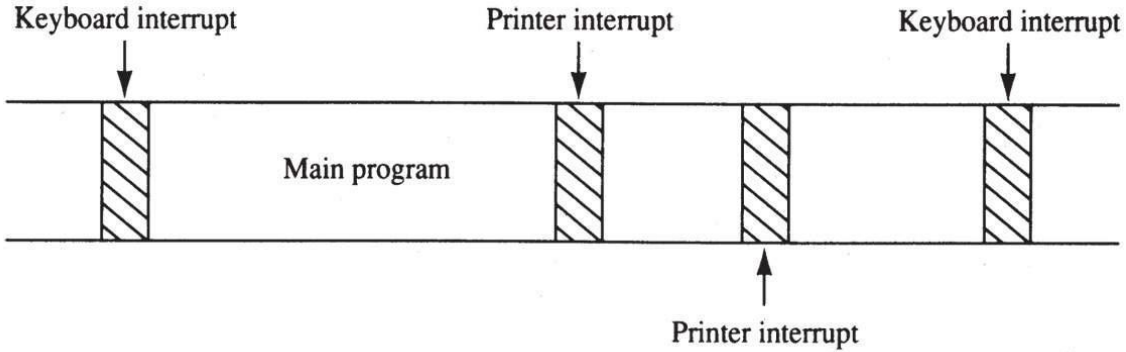
Interrupt

- An interrupt is a hardware or software generated change of flow within the system.
- The main purpose of the interrupt processing are:
  - Interrupts are useful when interfacing I/O devices at relatively low data transfer rates, such as keyboard inputs
  - It reduces the waste of time
  - It allows simultaneous execution of softwares

Interrupt : Example

- Interrupt processing allows the processor to execute other software while the keyboard operator is thinking about what to type next.
- When a key is pressed, the keyboard encoder debouncing a switch and puts out one pulse that interrupts the microprocessor.

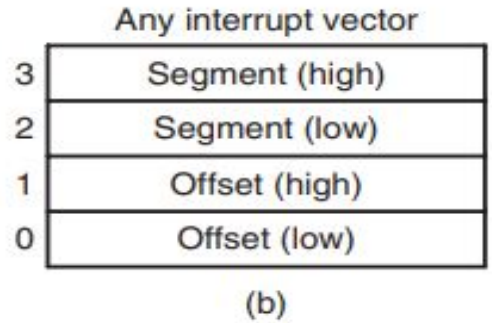
Figure 12–1 A time line that indicates interrupt usage in a typical system.



- a timeline shows typing on a keyboard, a printer removing data from memory, and a program executing
- the keyboard interrupt service procedure, called by the keyboard interrupt, and the printer interrupt service procedure is called by Printer interrupt
- each take little time to execute

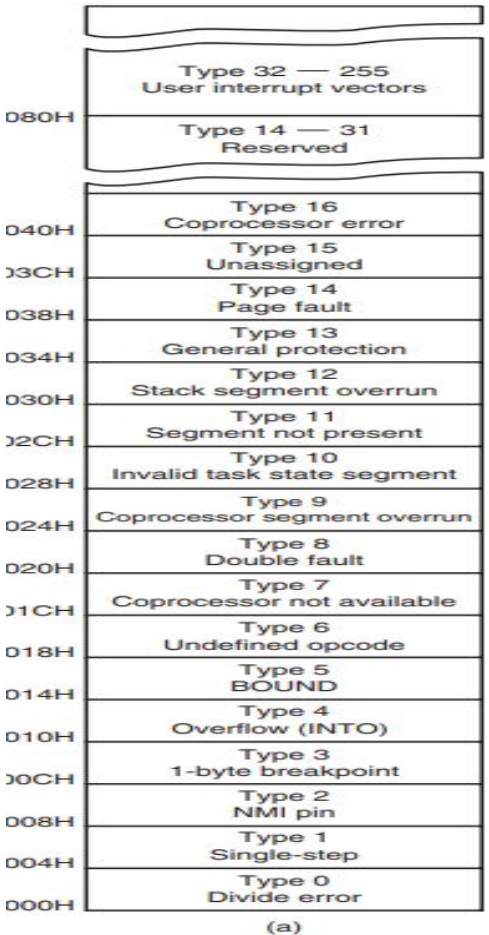
Interrupt Vectors

- An interrupt vector contains the address (segment and offset) of the interrupt service procedure.



- The **interrupt vector table** is located in the first 1024 bytes of memory at addresses 000000H–0003FFH.
- contains 256 different four-byte interrupt vectors

Interrupt Vectors



(a)

- Intel reserves the first 32 interrupt vectors
- the last 224 vectors are user-available
- each is four bytes long in real mode and contains the starting address of the interrupt service procedure.

Intel Dedicated Interrupts

- **Type 0**
  The **divide error** whenever the result from a division overflows or an attempt is made to divide by zero.
- **Type 1**
  Single-step or trap occurs after execution of each instruction if the trap (TF) flag bit is set.

- **Type 2**
  The **non-maskable interrupt** occurs when a logic 1 is placed on the NMI input pin to the microprocessor.
  – non-maskable—it cannot be disabled
- **Type 3**
  A special one-byte instruction (INT 3) used to store a breakpoint in a program for debugging

- **Type 4**
  Overflow is a special vector used with the INTO instruction. The INTO instruction interrupts the program if an overflow condition exists.
  - as reflected by the overflow flag (OF)

Type 5

- The **BOUND** instruction compares a register with boundaries stored in the memory.
- If the contents of the register are greater than or equal to the first word in memory and less than or equal to the second word, no interrupt occurs because the contents of the register are within bounds. if the contents of the register are out of bounds, a type 5 interrupt ensues

- For example, if the instruction BOUND AX,DATA is executed, AX is compared with the contents of DATA and DATA+1 and also with DATA+2 and DATA+3. If AX is less than the contents of DATA and DATA+1, a type 5 interrupt occurs. If AX is greater than the contents of DATA+2 and DATA+3, also type 5 interrupt occurs.

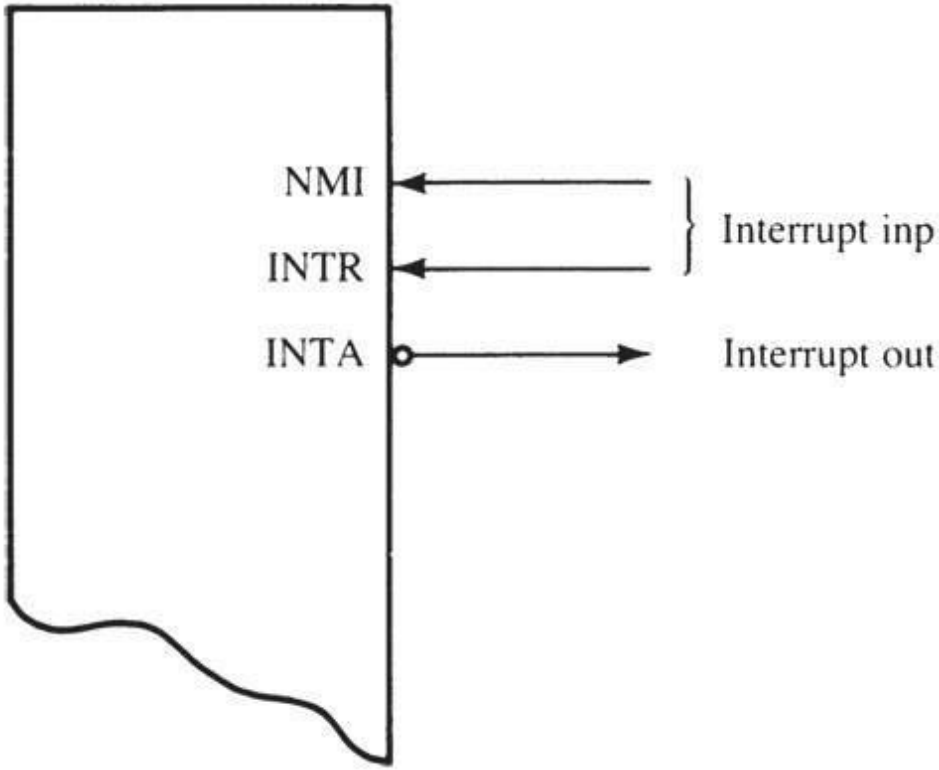
Interrupt Instructions: BOUND, INTO, INT, INT 3, and IRET

- **BOUND** : Check array against boundary
- **INTO:** The INTO instruction checks or tests the overflow flag (O). If O = 1, the INTO instruction calls the procedure whose address is stored in interrupt vector type number 4.
- **INT n:** INT n instruction calls the interrupt service procedure that begins at the address represented in vector number n. For example, INT 5 = 4 × 5 or 20 (14H). The vector for INT 5 begins at address 0014H and continues to 0017H
- **INT 3: Interrupt 3**
- **IRET:** used to return for both software and hardware interrupts.

12–2 HARDWARE INTERRUPTS

- The two processor hardware interrupt inputs:
  - non-maskable interrupt (NMI)
  - interrupt request (INTR)
- One interrupt output:
  - INTA

Figure 12–5 The interrupt pins on all versions of the Intel microprocessor.



The **non-maskable interrupt** (NMI):

- It is an input pin that requests an interrupt.
- The NMI pin must remain logic 1 until recognized by the microprocessor.
- This type of interrupts can not be avoided.
- The NMI input is often used for parity errors and other major faults, such as power failures.

- **Interrupt request input (INTR)**: It is used for requesting an interrupt. It must be held at a logic 1 level until it is recognized. INTR is set by an external event and cleared inside the interrupt service procedure .INTR is automatically disabled once accepted and re-enabled by IRET at the end of the interrupt service procedure
- **INTA:** The processor responds to INTR by pulsing INTA output. It is used to apply interrupt vector type number on data bus connections D_7–D_0.

Operation of a Real Mode Interrupt

- When the processor completes executing the current instruction, it determines whether an interrupt is active by checking:
  - (1) instruction executions
  - (2) single-step
  - (3) NMI
  - (4) coprocessor segment overrun
  - (5) INTR
  - (6) INT instructions in the order presented

- **If one or more are present**:
  - 1. Flag register contents are pushed on the stack
  - 2. Interrupt (IF) & trap (TF) flags **clear**, **disabling the INTR pin** and trap or single-step feature
  - 3. Contents of the code segment register (CS) are pushed onto the stack
  - 4. Contents of the instruction pointer (IP) are pushed onto the stack
  - 5. Interrupt vector contents are fetched and placed into IP and CS so the next instruction executes at the interrupt service procedure addressed by the vector

Interrupt Flag Bits

- IF: when IF is set, it *allows* the INTR pin to cause an interrupt. When IF is cleared, it *prevents* the INTR pin from causing an interrupt
-
- TF: when TF = 1, it causes a trap interrupt (type 1) to occur after each instruction executes .Trap is often called a *single-step*. when TF = 0, normal program execution occurs

– the interrupt flag is set and cleared by the STI and CLI instructions, respectively
– the contents of the flag register and the location of IF and TF are shown here

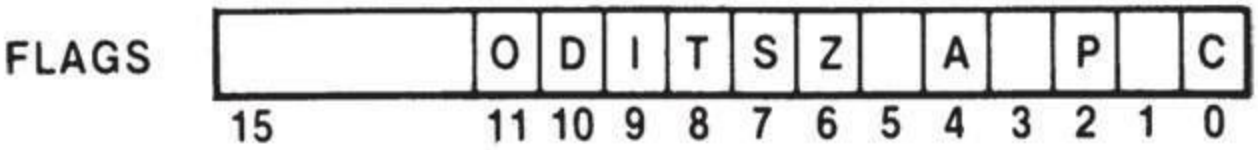


Figure 12–4 The flag register. (Courtesy of Intel Corporation.)

Priority Interrupt

- Whenever a peripheral is ready to transfer data , it is necessary to service its interrupt within a reasonable time.
  - Otherwise subsequent data may be lost
- **Faster peripheral require faster servicing**
- If two or more peripherals are ready at the same time, it is better to **service the faster one first** since the slow one can be made to wait a little while
- Multiple interrupt requests can be resolved by using a priority interrupt scheme
- A **hardware priority encoder** arbitrates between requests and sends a single value- that **represents the highest priority device**- to the CPU
- The interrupt request is formed by performing a **logical OR** of the peripherals IREQ lines.

Priority Interrupt

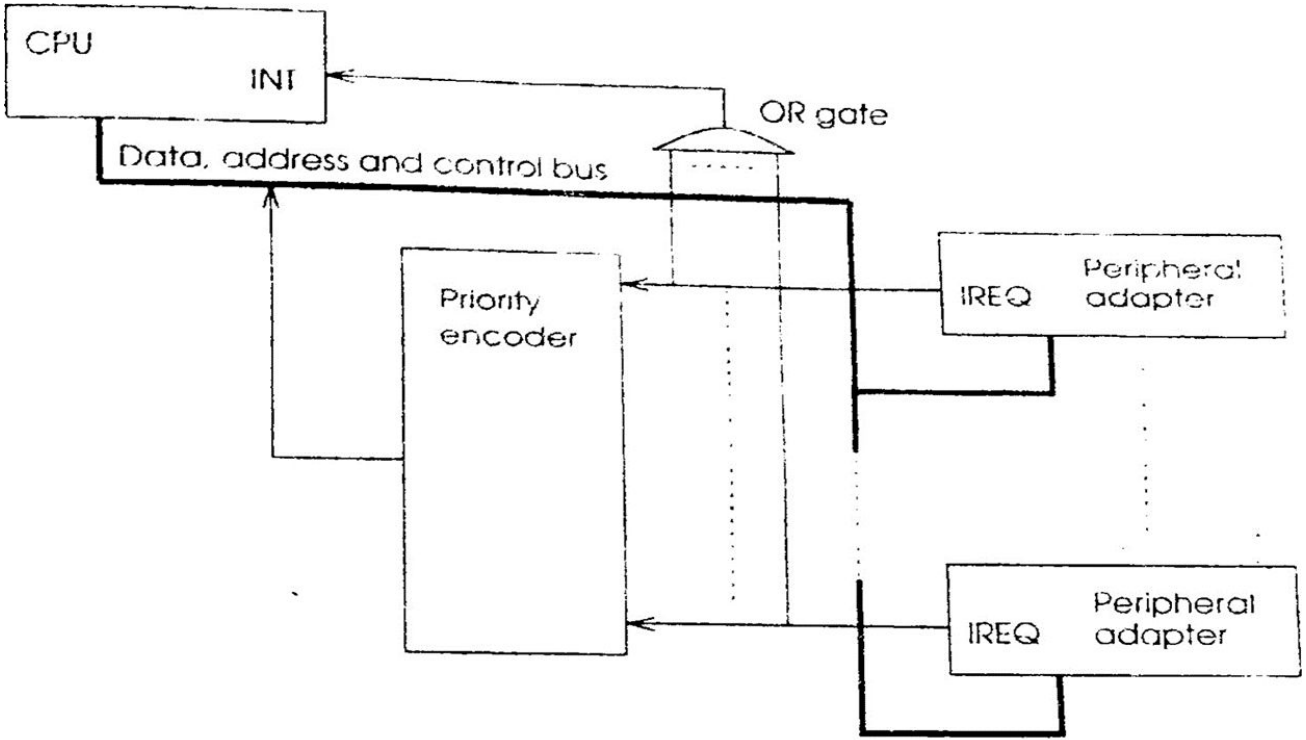


Figure 2.6 *Priority interrupts using a priority encoder*

Priority Interrupt Problem & Solution

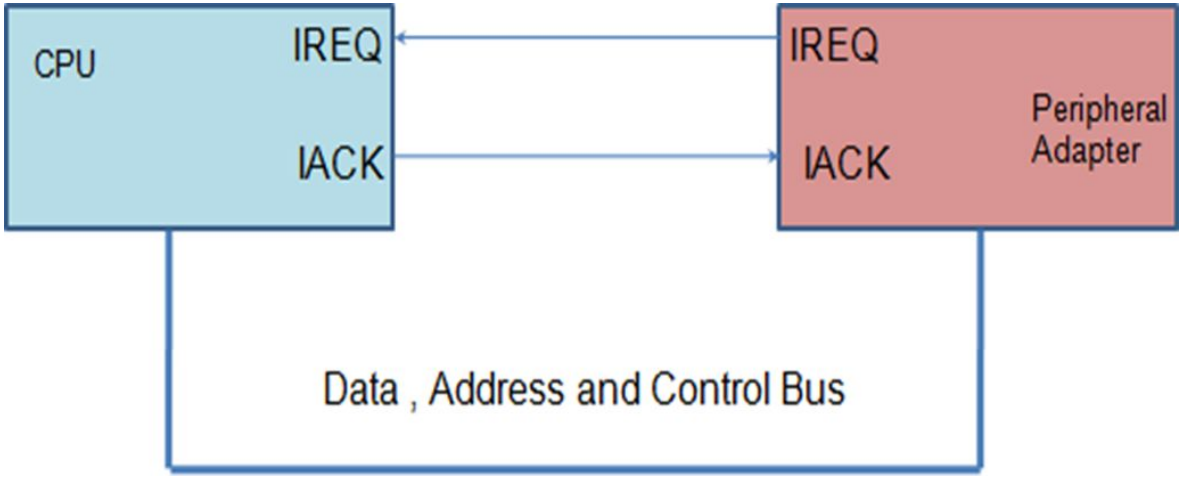
Problem

- Interrupt requests are assumed to **remain asserted until reset by instructions** in the service routine.
- It is not possible for another request to be seen until a request is de-asserted
- Data from a fast peripheral being lost while a service routine is getting around to clearing a low priority interrupt

Solution

- Most of the computers have a signal generated by the CPU that is returned to the peripheral as soon as the interrupt is detected- ***Interrupt acknowledge signal*** **(IACK).**
- This clears the interrupt request from that device and allows other devices to use the interrupt line.

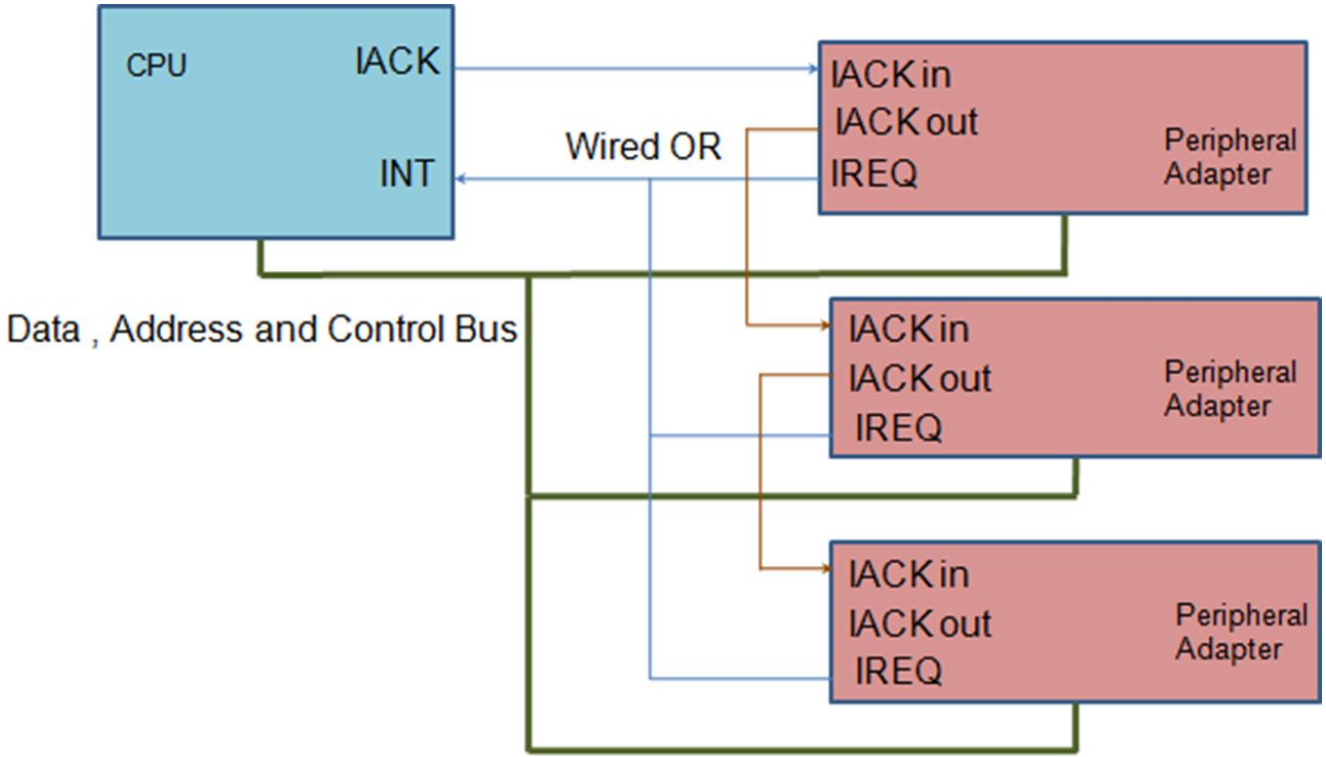
Interrupt with Acknowledgement



Priority Interrupt Using Daisy Chain

- Using an interrupt acknowledge enables the construction of priority interrupt scheme.
- CPU is able to **determine priority** not from the interrupt request but **by which device the acknowledge is sent to**.
- All the interrupt request lines are OR together.
- The **CPU IACK is connected directly to the highest priority device** so that if more than one request has been made the highest priority device sees it first.
- If it has not made a request, **it passes the IACK along to the next device**
- This continues down to the lowest priority device.
    - This receives an acknowledgement only if no other device has made a request.

Priority Interrupt Using Daisy Chain

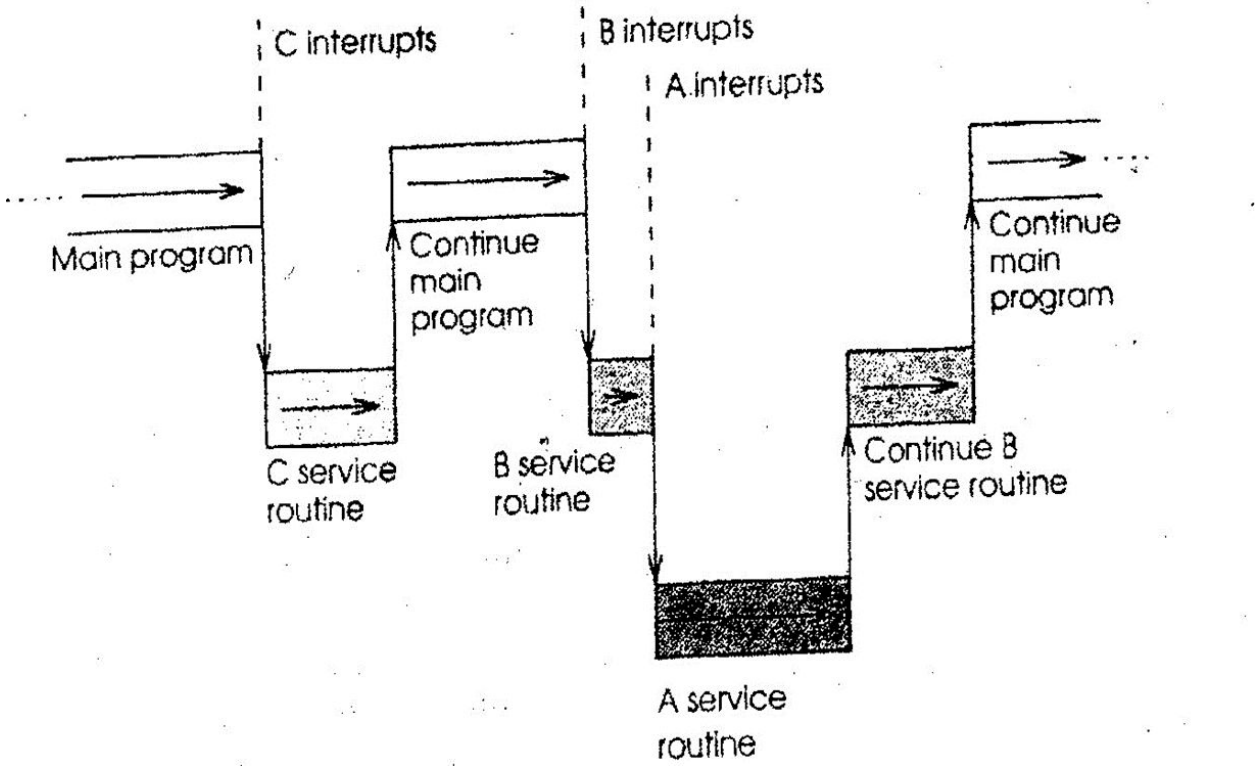


Nested Interrupt

- In order to ensure a fast response to a high priority interrupt it is possible to allow interrupts to interrupt interrupts.

  .

Nested Interrupt



Nested Interrupt

- Device C interrupt the main program, it's service procedure runs to the completion, returning to the main program.
- Device B similarly interrupt the main program but is,. In turn, interrupted by a higher priority device, A.
- Execution of the service procedure for A is nested within that for B
- When the service routine for device A complete the CPU returns to perform the service routine for B
- When it completes the main program continues.

.

Thank You